

DP Syntropy — Project Audit (2026-06-27)

Full review of n8n workflows (repo + live instance), the Supabase database (schema + live data + security), and the two HTML dashboards. Cleanup of the standalone LinkedIn tracker is included at the end.

1. Cleanup already done

The two standalone LinkedIn workflows ("Log LinkedIn Post", "Sync LinkedIn Clicks + Salesforce Leads") and their data table duplicated functionality already covered by the Syntropy attribution system (`actions.tracked_url` + `engagement_events` + GA4/Gmail measurement), and weren't wired into the WHEN Engine or any other part of the pipeline. As instructed, they've been retired:

- Workflow `TLR8FXQWAQDINjsb` (Log LinkedIn Post) — archived
- Workflow `KZh29d9tutF8CHTY` (Sync LinkedIn Clicks + Salesforce Leads) — archived
- Data table `z4ogiCZXGaKNa7zL` — renamed to "DEPRECATED - LinkedIn Post Tracker (unused, superseded by Syntropy pipeline)"

Note: the n8n MCP tools expose no hard-delete for workflows or data tables — archive/rename is as far as this can go without doing it manually in the n8n UI.

2. n8n workflows — documentation is badly out of date

`n8n-workflows/README.md` lists 5 workflows. The live instance (`designprodigy.app.n8n.cloud`) has **16**. Eleven are undocumented:

Real, active parts of the attribution system (should be documented, only one is exported to the repo):

- Calendly → Engagement (meetings) — `VLmJHUXyuvHVXdI7`, active
- Engagement → WHEN Signals — `grv4TN8pUrrW0xPa`, active
- Engagement Ingest (generic webhook at `/webhook/engagement`) — `fsPZuAbWdQb16kXm`, active
- GA4 Engagement Connector — `Y6nWWyM4SUNqUYtb`, active, exported to repo but missing from the README table
- Error Workflow — `rVr68buB9TD3qR5Y`, active (likely the global error handler)

Dead weight (inactive, safe to archive or ignore):

- 3× "Gate 1" duplicates — `X7Mz5pps1D00Q8oh`, `JtBTr7mZeg6abFTP`, `2o8MyNe2FLvOw40I`
- "My workflow 2" — `7VWKk6TsekchHAfg`
- "SC-AI Orchestration Backbone v2" — `Q691c0kw17ZEogoh` (different naming convention — looks unrelated to Syntropy)
- "GA4 → Engagement Feeder" — `20tIbI4ku7j35dma`, inactive, not exported (see next section — this one matters)

Active bug: duplicate-insert risk in GA4 Engagement Connector

The live, **active** "GA4 Engagement Connector" workflow:

- Hardcodes GA4 property ID `542471069` instead of reading `app_config.ga4_property_id`
- Has **no idempotency guard** on its `INSERT INTO engagement_events`

- Stamps `occurred_at` with `now()` instead of the actual GA4 event date

Meanwhile the **inactive** "GA4 → Engagement Feeder" is the correctly-built version: reads `ga4_property_id` from `app_config` (with the same fallback), uses a 3-day lookback, filters out `(not set)`/zero-session rows, and has a `NOT EXISTS` idempotency check keyed on campaign+date.

Right now this is harmless because `engagement_events` has 0 rows (no real tracked traffic yet — `tracking_base_url` is unset). But the moment real campaign traffic starts, the active Connector will insert a fresh duplicate row for every account on every scheduled run. **Recommendation: deactivate "GA4 Engagement Connector", activate "GA4 → Engagement Feeder".**

Minor: placeholder email left in Action Approval Handler

"Send Action to Rep" (Gmail node) still points to `rep@yourcompany.com`. The equivalent nodes in Brief Approval Handler correctly use `honlamchia@gmail.com`.

3. Supabase — `schema.sql` is stale, and there's an undocumented subsystem

Schema drift (repo `supabase/schema.sql` vs. live DB)

- `actions` (live) has 3 columns missing from `schema.sql`: `target`, `priority`, `reason`
- `engagement_events` (live) has 1 column missing from `schema.sql`: `forwarded boolean default false` — this is actively used by "Engagement → WHEN Signals" to track what's been forwarded, so the *workflow* is correct, the *schema file* is what's behind.
- Six dashboard views exist live but aren't in `schema.sql`: `dashboard_gate1_breakdown`, `dashboard_vetoed_companies`, `account_score`, `dashboard_top_accounts`, `dashboard_engagement_funnel`, `dashboard_action_type`.

Practical effect: following the README's own "Setup (new environment): run `supabase/schema.sql` against a fresh project" instructions would **not** reproduce the current system.

Undocumented subsystem: pgvector / RAG

The live database also contains a `vector` Postgres extension, a `match_documents` function, and tables `documents`, `agent_inputs`, `agent_inputs_bak` — none of which appear anywhere in the README or `schema.sql`. This looks like a separate embeddings/RAG component (possibly tied to the unrelated-looking "SC-AI Orchestration Backbone v2" workflow noted above). Worth a decision: document it as part of the system, or confirm it's dead and drop it.

There are also unexplained backup tables sitting in the public schema: `pipeline_runs_bak`, `pipeline_errors_bak` — these showed up in the security scan too (see below).

Security — this is the most important section

Run via Supabase's advisor tooling against project `fqsqoxsosavjhdsvfek`:

- ~25 dashboard/console views use `SECURITY DEFINER` (ERROR level) — they run with the view creator's privileges rather than the querying user's, bypassing RLS.

- **RLS is fully disabled** (ERROR level) on `actions`, `outcomes`, `engagement_events`, `documents`, plus the backup tables `pipeline_runs_bak`, `pipeline_errors_bak`, `agent_inputs_bak`.
- This matters specifically because **both dashboard HTML files embed the Supabase anon key directly in client-side JavaScript** (`SUPABASE_KEY` constant), and the README already flags this as "internal only, don't host publicly." With RLS disabled on those tables, anyone who has that key — which is sitting in plain text in a static HTML file — can read or write those tables directly via the Supabase REST API, not just through the dashboard's intended views.
- `app_config` has an "always true" RLS policy granting **UPDATE to anon** (`app_config_anon_update`). This is intentional — it's what powers the dashboard's Settings tab — but it means anyone with the anon key can rewrite `tracking_base_url`, `ga4_property_id`, or `when_engine_url` directly, bypassing the UI entirely.
- I verified this last point concretely: the "shared" dashboard (`pipeline-control-tower-shared.html`) disables the Settings inputs and hides the Save button in the HTML, and its caption literally says "Read-only shared view." But it embeds the **exact same anon key** with the **exact same UPDATE grant**, and the page's `saveSettings()/patchConfig()` JavaScript functions are still present in the file. The "lock" on the shared copy is cosmetic (a disabled `<input>` attribute) — it is not a real permissions boundary. Anyone who opens dev tools, or just sends a raw **PATCH** request with the embedded key, can still rewrite `app_config` from the "read-only" copy.
- Separately: the brief/action approval links the dashboard opens (`/webhook/brief-approval?id=...&action=approve`, `/webhook/action-approval?id=...&action=approve`) are **plain unauthenticated GET requests** — I checked both workflow JSONs and neither webhook node has any auth configured. Anyone with (or guessing) a small integer `id` can approve or reject any brief or action with no credentials.
- Lower severity: `pipeline_runs` and `pipeline_errors` have RLS enabled but zero policies (INFO — effectively locked down by default, which is fine and is why the SECURITY DEFINER views are doing the actual data exposure work for these two tables).

Net read: the security model currently rests entirely on "the anon key is hard to find," not on RLS or auth. That's consistent with an internal-tool design but is worth being deliberate about, especially since the "shared" dashboard is explicitly meant to be handed to other people.

2026-06-28 — Remediated. Three changes, keeping the static-HTML/no-login model intact:

1. **RLS + grants on the exposed tables.** Enabled RLS on `actions`, `outcomes`, `engagement_events`, `documents`, `pipeline_runs_bak`, `pipeline_errors_bak`, `agent_inputs_bak` (all previously had RLS fully disabled), and revoked all anon/authenticated grants on these plus `pipeline_runs/pipeline_errors` (already RLS-on-no-policy, but still had stray grants that would've become exploitable the moment a policy was added). Default-deny is safe here because every legitimate read goes through a `SECURITY DEFINER` view (bypasses base-table grants/RLS via the view owner) and every legitimate write goes through n8n's Postgres credential, which connects as `postgres` — confirmed `rolbypassrls = true`, so unaffected by RLS. Fixed `agent_inputs`'s `service_role_all` policy, which was named for `service_role` but actually granted to `{public}` (i.e. anon + authenticated too) — replaced with a policy genuinely scoped to `service_role`.
2. **Scoped the `app_config` anon UPDATE.** The policy now only matches `key IN ('tracking_base_url', 'ga4_property_id')` — the two keys the dashboard's Settings tab actually edits — and the anon grant is now column-level (`UPDATE (value)` only, matching what

`patchConfig()` actually sends). This closes a real gap: `when_engine_url` (controls where rejected briefs/actions get forwarded) was previously anon-writable via a raw PATCH even though no UI ever exposed that. Verified live: anon can still update `tracking_base_url/ga4_property_id`, cannot touch `when_engine_url`.

3. **Signed the approval webhooks.** `/webhook/brief-approval` and `/webhook/action-approval` now require an HMAC-SHA256 `sig` query param (scoped per-webhook, computed server-side at link-generation time and re-verified on receipt with `crypto.timingSafeEqual`); requests with a missing/wrong signature get a 403 instead of reaching the approve/reject logic. Also fixed: the `reason` query param (used in `Action Approval Handler's Mark Cancelled` Postgres node, which builds its query via string interpolation) is now whitelist-sanitized to `[a-zA-Z0-9 _-]` before use — closing the SQL-injection path that existed in that node. Verified via `test_workflow`: valid signatures route to the approve/reject path, invalid ones route to the 403 responder, and a SQL-injection-shaped `reason` gets stripped clean before it could reach the vulnerable node.

Residual, out of scope for this pass: the ~25 `SECURITY DEFINER` console/dashboard views (ERROR level) are the intended access mechanism in this no-auth design, not a bug — removing `SECURITY DEFINER` from them would break the dashboard's reads entirely. `match_documents`'s mutable `search_path` and the `vector` extension living in `public` are both pre-existing, low-severity, and unrelated to the access-control gaps above.

2026-06-28 — second pass, found while building the campaigns subsystem (below). Every pre-existing dashboard/console view (25 of them) still had full `INSERT/UPDATE/DELETE/TRUNCATE/REFERENCES/TRIGGER` granted to `anon/authenticated` — only `app_config` was correctly `SELECT`-scoped. Most of these views are joins/aggregates/`GROUP BY/LIMIT`, so Postgres's auto-updatable-view rules block writes through them regardless of grant. But `feedback_for_tabita` and `feedback_for_engine` are plain single-table `SELECT ... FROM actions WHERE ... ORDER BY ...` views with no aggregation, `GROUP BY`, `DISTINCT`, or `LIMIT` — which makes them genuinely auto-updatable. With the broad grant in place, `anon` could `INSERT/UPDATE/DELETE` directly against `actions` through either view, fully bypassing the RLS lockdown from the first pass. Fixed by revoking all and re-granting `SELECT` only on all 25 views (migration `lock_dashboard_views_to_select_only`). Verified: `role_table_grants` now shows `{SELECT}` only on every dashboard/console view, and a follow-up `get_advisors` security scan came back with no new findings — only the same pre-existing/accepted `security_definer_view`, `rls_enabled_no_policy`, `function_search_path_mutable`, and `extension_in_public` items.

New subsystem: campaigns & cadences (2026-06-28)

Added multi-step cadence tracking on top of the existing single-shot `actions` table, per the roadmap discussion about evolving the Action Layer into multi-touch outreach:

- **Tables:** `campaigns` (one per account/run, `status` active/completed/cancelled) and `cadence_steps` (ordered steps per campaign — `step_number`, `delay_days`, `scheduled_for`, the same action-shape columns as `actions` — `account/action_type/target/reason/hypothesis/predicted_metric/predicted_direction/threshold/window_days/utm_campaign/tracked_url/outreach_draft` — plus `status` pending/recommended/executed/measured/cancelled/skipped and `action_id` once a step is promoted to a real action). `actions` gained a `campaign_id` FK back to `campaigns`.

- **Trigger:** `trg_sync_cadence_step_status` (AFTER UPDATE on `actions`) calls `sync_cadence_step_status()`, which mirrors an action's status onto its originating `cadence_steps` row whenever it changes — so approving/cancelling an action in the existing approval flow automatically updates the cadence view without any new approval-handler logic.
- **Workflow wiring:** Brief Approval Handler now creates a `campaigns` row + its `cadence_steps` on approval (instead of just one `actions` row). A new workflow, **Cadence Scheduler** (`LcZx5U9dVm0waGQ1`, daily 06:30 schedule trigger), finds due steps (`scheduled_for <= now()` and `status='pending'`), inserts each as a real `actions` row, marks the source `cadence_steps` row `recommended`, and emails an HMAC-signed approval link per step using the same signing scheme as the other approval emails — so due cadence steps flow into the existing approval inbox rather than a separate one.
- **Views:** `dashboard_campaigns` (per-campaign rollup: total/completed/awaiting-approval/upcoming/cancelled/skipped step counts, next step due) and `dashboard_cadence_steps` (flat per-step list with account/step/delay/scheduled time/status). Both **SECURITY DEFINER**, both granted `SELECT` only to `anon/authenticated` (caught and fixed an initial overgrant on these two — see the security note above).
- **Dashboard:** new "Campaigns" tab in both `pipeline-control-tower.html` and `-shared.html` — 4 KPI cards plus a campaigns table and a cadence-steps table, both backed by the two views above.

End-to-end test of the cadence flow (2026-06-28)

Ran the full loop against a disposable test account (`ZZZ-QA-TEST-CADENCE`, deleted afterward) rather than trusting that "saved" meant "live":

Found and fixed: Brief Approval Handler's campaign wiring was never published.

`get_workflow_details` showed the workflow's draft (`versionId`) and its live `activeVersionId` had diverged — the draft had the new `Insert Campaign/Due Now?`/cadence-step nodes, but the `active` version n8n actually runs on a real webhook hit was still the old single-shot logic. First test call confirmed it: approving a test brief inserted 5 `actions` rows with `campaign_id = null` and created zero `campaigns/cadence_steps` rows — exactly what the old code would do. Fixed with `publish_workflow` (promotes the current draft to active); same staleness existed on Action Approval Handler so it was published too, though that one's draft/active difference was cosmetic (extra `RETURNING` columns, disabled Salesforce nodes) rather than a behavior bug. Re-ran the same test after publishing and got the correct result: 1 `campaigns` row, 5 `cadence_steps` (step 1 due-now → immediately promoted to a real `action`; steps 2–5 → `pending`, scheduled 2/5/7/10 days out).

Then verified the rest of the chain: approved the immediate step's action through Action Approval Handler → `actions.status` flipped to `executed` → the `sync_cadence_step_status` trigger correctly mirrored that onto the originating `cadence_steps` row with no extra workflow logic. Backdated one of the still-`pending` steps to be "due" and ran Cadence Scheduler → it found the due step, inserted a real `actions` row, marked the source step `recommended` with `action_id` set, and built an approval email with HMAC-signed approve/reject links — recomputed both signatures independently and they matched exactly what Action Approval Handler's own `Verify Signature` node expects. `get_advisors` (security) afterward showed no new findings, only the same pre-existing/accepted set.

Takeaway: when editing a published n8n workflow through the SDK/MCP tools, the edit lands in the draft and does *not* automatically go live — `publish_workflow` is a required, separate step. Worth checking on any future workflow edit, not just this one.

Live data snapshot

- **pipeline_runs**: 384 rows, 2026-06-10 → 2026-06-26. 281 briefs generated, 71 human-approved (~18.5%), 4 vetoed at Gate 1.
- **actions**: 50 rows. Only 3 have ever reached **executed** (all **content_send**), 0 have reached **measured**. 14 cancelled. The remaining 33 sit at **recommended** across email/invite/landing_page/linkedin_connect/linkedin_engage/linkedin_post/content_download/content_send.
- **outcomes**: 0 rows — nothing measured yet (consistent with only 3 actions ever executed).
- **pipeline_errors**: 13 rows, **all unresolved**. 11 of 13 trace to a "Disqualifier" node and 2 to "Gate 1 Pipeline" — neither name exists in the 5 repo-exported workflows, so these are almost certainly leftover from the retired "Gate 1" duplicate workflows or the un-exported monolith, not live issues. Worth marking resolved or confirming they're dead.
- **engagement_events**: 0 rows — expected, since **tracking_base_url** and **when_engine_url** are both unset in **app_config**, so the whole measurement/forwarding loop is a designed no-op until those are configured.

New subsystem: Salesforce sync prototype (2026-06-28)

Follow-up to a strategic question about repositioning the project as an orchestration layer over CRM/MAP tools (Salesforce, Marketo) rather than a standalone campaign engine. Scoped down to a concrete, testable prototype:

- **Decision: Salesforce only, Supabase stays source of truth.** Salesforce was prototyped because a working credential already exists in n8n; Marketo was scoped out entirely — no org, no credentials, no native n8n node. Supabase's **campaigns/cadence_steps** remain authoritative; Salesforce is a one-way synced execution layer underneath them, not a replacement. Nothing reads back from Salesforce into Supabase.
- **Mechanism discovery:** n8n's Salesforce node has no first-class "Campaign" resource (only Account/Contact/Lead/Opportunity/Case/Task/etc., plus a generic **customObject** resource meant for true **__c** custom objects). Tested empirically against the live Salesforce credential: passing the standard object API name "**Campaign**" directly into the **customObject** field works for **create** — confirmed via a disposable probe workflow that created a real Campaign (**7010Q00000cm87LYAQ**) and then deleted it through the same mechanism's **delete** operation (**success:true**). Field values are set via **customFieldsUi.customFieldsValues: [{fieldId, value}]**, where **fieldId** also accepts standard field API names (**Name, Status, Type**) as free text. This closes out the need for an HTTP Request fallback (which would have needed the Salesforce instance base URL).
- **Schema (migration add_salesforce_sync_fields):** **campaigns** gained **salesforce_campaign_id**, **salesforce_sync_status** (**pending/synced/error/skipped**, default **pending**), **salesforce_synced_at**, **salesforce_sync_error**. **actions** gained a reserved, currently-unpopulated **salesforce_task_id** column (for a likely future per-action Task sync, tied to the existing disabled Find-Account/Create-Task nodes in Action Approval Handler) — added now to avoid a second migration later.
- **Workflow wiring:** Brief Approval Handler's existing **Insert Campaign** node got a second, parallel output branch (the original **Insert Campaign → Recommend Action → ...** cadence chain is untouched): **Create Salesforce Campaign** (Salesforce node, **customObject/create/Campaign, onError: continueErrorOutput**) → success branch **Mark Salesforce Synced** (Postgres UPDATE,

sets `salesforce_campaign_id/synced/salesforce_synced_at`), error branch `Mark Salesforce Sync Error` (Postgres UPDATE, sets `error` + the Salesforce error message via `queryReplacement` to avoid string-concatenation injection). A Salesforce failure can't break brief approval or cadence-step creation — it's a sibling branch, not a step in the existing chain.

- **Caught while building:** the new `Mark Salesforce Synced` node's query was missing the leading `=` — `n8n` requires to treat a string as an expression (`MISSING_EXPRESSION_PREFIX`) — fixed before testing. (Two pre-existing, already-proven-working nodes — `Mark Brief Approved`, `Mark Brief Rejected` — have the same missing-prefix pattern and were deliberately left alone; touching working production nodes outside this change's scope wasn't worth the risk.)

End-to-end test of the Salesforce sync (2026-06-28)

Same disposable-test-data discipline as the cadence flow test above, plus the same staleness check it had already caught once before:

Published Brief Approval Handler first (`publish_workflow`) — confirmed via `get_workflow_details` that the draft and active version had diverged again (new Salesforce branch existed only in the draft), so this was a required step, not a formality.

Inserted a disposable `pipeline_runs` row (`ZZZ-QA-TEST-SALESFORCE`), computed a valid HMAC `approve` signature, and hit the real production webhook. Result: the existing cadence chain behaved exactly as before (1 `campaigns` row, 1 `actions` row at `recommended`, 5 `cadence_steps`) — fully unaffected by the new branch — and in parallel, a real Salesforce Campaign was created (`7010Q00000cmOTmYAM`) with `campaigns.salesforce_sync_status` flipping to `synced` and `salesforce_synced_at` populated, `salesforce_sync_error` null.

Cleaned up: deleted the Salesforce Campaign through a second disposable probe workflow (same `customObject/delete` mechanism), then archived that workflow; deleted the test `cadence_steps/actions/campaigns/pipeline_runs` rows from Supabase. Verified zero rows remain under any of those IDs.

Deliberately not built: Campaign Members. A Salesforce Campaign is just a shell — real tracking (response logging, member status, rollup counts) needs Contacts/Leads attached as Campaign Members. Checked the live data first: every `pipeline_runs.raw_payload.buying_group` entry across the whole table has `name/title/focus_area/status` but **no email field at all** (`raw_payload::text ILIKE '%email%'` → zero rows), and many entries are role placeholders (`name: "Group CEO"`, `status: "Needs enrichment"`) rather than real people. Asked the user how to handle this; they chose to **hold off on Campaign Member sync for now** rather than push email-less Leads or build a Supabase-side enrichment step. Revisit if/when the WHEN Engine starts including contact emails, or if a manual enrichment step gets built.

Lead / Campaign Member sync — unblocked, and Task attach fixed (2026-06-28)

Picked back up the "deliberately not built" gap above, in Action Approval Handler this time (not Brief Approval Handler). Re-checked the email gap first — still zero `email` matches anywhere in `raw_payload.buying_group` — and worked around it rather than waiting:

- **Discovery: no `customObject` workaround needed here.** Unlike Campaign, `n8n`'s Salesforce node has a first-class `lead.addToCampaign` operation — the `CampaignMember` equivalent — so this is a native operation, not the `customObject` trick used for Campaign creation.

- **Dedup without email.** Lead's native `upsert` needs a custom External ID field this org doesn't have, so dedup is manual: SOQL `search/query` on `LastName` + `Company` before `create` (mirrors the same search-then-act shape as the old disabled `Find Account` node it replaces). Only `buying_group` entries with `status: "CONFIRMED"` are processed — role placeholders (`name: "Group CEO", status: "Needs enrichment"`) are filtered out in the new `Extract Confirmed Contacts` Code node.
- **Task-attach fix, no extra lookup needed.** The old (disabled) path found the Task's target via a fuzzy `Find Account` SOQL search on Account name. Replaced it: `Mark Executed`'s own `RETURNING` clause now pulls `campaign_id` plus two correlated subqueries (the run's `raw_payload`, and the campaign's `salesforce_campaign_id`) — so `Create a task` (now enabled) points `whatId` straight at the already-synced Campaign, and `Find Account` was deleted outright rather than left disabled.
- **New branch, doesn't block the rep email:** `Mark Executed` → `Extract Confirmed Contacts` (Code) → `Find Existing Lead` (SOQL search) → `Lead Found?` → either `Add Existing Lead To Campaign` or `Create Lead` → `Add New Lead To Campaign`. Every Salesforce node in this branch and `Create a task` is `onError: continueRegularOutput` — a Lead/Task/CampaignMember failure can't block "Send Action to Rep"/"Respond Approved".
- **Caught while building:** `update_workflow`'s validator flagged `Mark Executed` (the node being edited) and `Mark Cancelled` (pre-existing, untouched by this change — same latent issue as the `Mark Brief Approved/Mark Brief Rejected` pair caught in the section above) both missing the leading `= n8n` needs to treat a string as an expression. Fixed both this time, since both queries were already being touched in this work area.
- **Verified, scoped down from a full live Salesforce E2E:** `publish_workflow` confirmed the draft/active divergence pattern again (3rd time this has bitten a workflow edit in this project — see the "Takeaway" above). Ran a `test_workflow` execution with a valid HMAC-signed disposable payload (`ZZZ-QA-TEST-LEADSYNC`, one `CONFIRMED` contact + one `Needs enrichment` placeholder) — confirmed the signature check passes for real, `Approved?` routes correctly, `Extract Confirmed Contacts` correctly drops the placeholder and keeps only the confirmed contact, and `Lead Found?` routes to the right branch. Separately ran the new `Mark Executed` correlated-subquery shape as a read-only `SELECT` against the live schema to confirm the column names/joins are valid. **Not done:** an actual live Salesforce Lead/CampaignMember create+delete (like the Campaign create+delete probe above) — `campaigns/actions` currently have zero rows with `campaign_id` populated in production, so there's nothing live to safely exercise this against yet without fabricating disposable Salesforce records from scratch. Worth a real spot-check the first time a cadence-driven action actually reaches `executed`.

Held off (2026-06-28, same day): user asked to pause this before it runs against real data. Disabled the 5 nodes that actually call Salesforce — `Create a task`, `Find Existing Lead`, `Create Lead`, `Add Existing Lead To Campaign`, `Add New Lead To Campaign` — via `disabled: true`, then published, leaving the rest of Action Approval Handler (status updates, rep email, reject-forwarding) running exactly as before. Chose this over a full `unpublish_workflow` because that would have also stopped the webhook from responding at all, breaking the parts that already worked. While disabling, the draft/active divergence bug above bit again — the draft copies of these 5 nodes (plus `Send Action to Rep`, untouched by this change) had silently lost their `operation/resource` discriminator fields and, on `Mark Executed/Mark Cancelled`, the leading `=` — all present and correct in the already-published active version, missing in the draft. Re-added them to the draft from the known-good active version before publishing, so the disable didn't ship a regression alongside it. **Current state: webhook live, core approve/reject flow unaffected, all 5 Salesforce-writing nodes disabled** — re-enable by clearing `disabled` on those 5 nodes once reviewed.

New subsystem: campaign templates (2026-06-28)

Follow-up to the user's "I don't own a website to turn on real tracking, I'm just an intern" constraint — instead of a manual template picker, made cadence selection automatic from signals already on the brief, via `AskUserQuestion` confirming "automatic by brief signal" over a manual dropdown or single default.

- **Table:** `campaign_templates` (`id`, `key`, `name`, `priority`, `match_urgency_in text[]`, `match_coverage_in text[]`, `match_stage_contains text`, `steps jsonb`). NULL on any match column means wildcard. Seeded 4 rows: `urgent_fast_track` (priority 10, 3 steps), `narrow_coverage_nurture` (15, 4 steps), `cold_outreach` (20, 5 steps), `warm_reengage` (99, all match columns NULL — catch-all, 5 steps). Each step: `{action_type, delay_days, reason}`.
- **Workflow wiring (Brief Approval Handler):** new Postgres node `Select Campaign Template` runs `... ORDER BY priority ASC LIMIT 1` with `$1/$2/$3` bound to the brief's `when_urgency/coverage_label/when_stage`, sits between `Get Config` and `Insert Campaign`. `Insert Campaign` now also writes `template_key`. `Recommend Action` was rewritten to map the matched template's `steps` into the action plan instead of a hardcoded COLD/not-COLD `plan` array — same output shape (`campaign_id/run_id/account/target/action_type/priority/reason/delay_days/scheduled_for/hypothesis/predicted_metric/utm_campaign/tracked_url/outreach_draft/status`), so nothing downstream changed. Falls back to a 2-step safety net if no template matches (shouldn't happen — `warm_reengage` is a wildcard).
- **Verification:** SQL matching logic checked independently against the 4 seeded rows for all 4 signal permutations (`urgent` → `urgent_fast_track`, `NARROW coverage` → `narrow_coverage_nurture`, `COLD stage` → `cold_outreach`, everything else → `warm_reengage`) before trusting n8n's test harness, since `test_workflow` pins the Postgres node's own output and can't exercise the SQL itself. Then ran a full `test_workflow` (cold-stage signal, real HMAC-signed approve payload, `Select Campaign Template`'s output pinned to the real `cold_outreach` row) and inspected execution data on `Recommend Action/Due Now?`/the two insert nodes — confirmed 5 correctly-enriched actions, correct per-action-type metric/channel/tracked_url, correct `theMove` override on the email step only, and correct immediate (`delay_days=0`) vs. scheduled branching.
- **Hit the draft/active divergence bug twice** (see `n8n_draft_active_divergence` memory) while wiring this in: `Insert Campaign` had silently lost its `operation: "insert"` in draft, and `Create Salesforce Campaign` had silently lost both `operation: "create"` and was showing `disabled: true` in draft while live/active had it enabled — the second one would have silently turned off live Salesforce Campaign sync if published without catching it. Both fixed via `updateNodeParameters(merge)/setNodeDisabled` before publish; confirmed clean via a final draft-vs-active diff.

Closed the measurement gap for "vibes" signals + `quick-log.html` (2026-06-28)

Four `predicted_metric` values — `linkedin_accept`, `profile_visit`, `registration`, `content_download` — had existed on actions since the campaign-templates build but were never evaluated by Action Outcome Measurement; only `page_visits` (GA4) and `reply_or_meeting` (Gmail) had a measurement path. Closed the gap and gave it a real front-end rather than requiring raw JSON POSTs to `/webhook/engagement`:

- **Workflow wiring (Action Outcome Measurement, `RMPUPcf2o0ltJJVeG`):** `Get Due Actions` widened to include the 4 metrics above. New branch: `Route by metric` → `Is Reply Metric?` (IF) → `Check Engagement Events` (Postgres, sums `engagement_events.value` joined on `action_id` OR `campaign` within the action's window) → `Evaluate vs Engagement Events` (Code, same met/not-met threshold

logic as the GA4/reply branches) → **Insert Outcome (engagement)** → **Mark Measured (engagement)**.

- **Companion change (Engagement Ingest, fsPZuAbWdQb16kXm):** **Normalize** and **Insert Event** now pass through an optional **metadata** (jsonb) field, so a manual log can carry a free-text note (e.g. who replied, what they said).
- **New front-end:** **dashboards/quick-log.html** — pick from "open" (**status='executed'**) actions, choose the signal that happened, optionally add a note, click "Log it" → POSTs to the existing **/webhook/engagement** endpoint. Backed by two new views: **console_open_actions** (**status='executed'**; distinct from the existing **console_pending_actions**, which is **status='recommended'** only) and **console_recent_engagement** (last-15 confirmation feed).
- **Verification:** tested the new measurement branch in isolation (**test_workflow** with pinned engagement data) before publishing, then ran a genuine end-to-end test against a disposable action (**ZZZ-QA-TEST-ENGAGEMENT-E2E**) through both **real, published** workflows via **execute_workflow** (production mode, not **test_workflow**) — confirmed a real **engagement_events** row, a real **outcomes** row, and **actions.status** flipping to **measured**. All disposable rows deleted afterward; verified zero residual.

Security: third occurrence of the default-ACL / auto-updatable-view leak (2026-06-28)

Found while creating the two views above — same bug class as the **feedback_for_tabita/feedback_for_engine** incident in the "second pass" note earlier in this doc, but this is a **separate, still-unfixed root cause**: that prior fix revoked the over-grant on the *existing* views at the time, it did not remove the default ACL rule that keeps producing this on every *new* view.

- **Root cause confirmed via pg_default_acl:** this project's **public** schema has **ALTER DEFAULT PRIVILEGES FOR ROLE postgres IN SCHEMA public GRANT ALL ON TABLES TO anon, authenticated**. Every newly created relation — table or view — gets full **INSERT/UPDATE/DELETE/TRUNCATE/REFERENCES/TRIGGER** for **anon/authenticated** automatically at creation time, regardless of any **GRANT SELECT** issued afterward (the explicit grant only adds to the default-ACL grant, it doesn't replace it).
- **Why it's exploitable:** **console_open_actions** and **console_recent_engagement** are both plain single-table **SELECT ... FROM x WHERE ... ORDER BY ...** views with no join/aggregate/**DISTINCT/GROUP BY** — exactly the shape Postgres treats as auto-updatable. So **anon** could **INSERT/UPDATE/DELETE** straight through either view into the locked-down base tables **actions/engagement_events**, fully bypassing their RLS lockdown from the first security pass.
- **Fix applied:** migration **lock_down_new_console_views_select_only** — explicit **REVOKE ALL ... FROM anon, authenticated, PUBLIC** on both views, then re-**GRANT SELECT**. Verified via **information_schema.role_table_grants** before (full CRUD) and after (SELECT only).
- **Not yet fixed: the default ACL rule itself.** This is the third time this exact class of bug has shipped in this project (1st: the original 25 dashboard views; 2nd: **feedback_for_tabita/feedback_for_engine**; 3rd: these two). Each time the fix has been per-view (revoke-then-grant), not a removal of the **ALTER DEFAULT PRIVILEGES** rule that keeps causing it. **Recommended next step:** either drop the default-ACL rule entirely (**ALTER DEFAULT PRIVILEGES FOR ROLE postgres IN SCHEMA public REVOKE ALL ON TABLES FROM anon, authenticated**) and grant **SELECT** explicitly per-view going forward, or add a standing checklist item — any future **CREATE VIEW/CREATE TABLE** in this schema must be followed by an explicit grant audit via **role_table_grants** before being wired into a public-facing

dashboard. Documented in `supabase/schema.sql`'s grant-block comment and in memory (`supabase_default_acl_leak`).

New subsystem: channel recommendation AI (Phase 5, 2026-06-29)

The campaign-templates work above (2026-06-28) made cadence *selection* automatic but the cadence *content* was still 100% static — whichever template matched dictated the exact channel sequence for every account, regardless of that account's actual engagement history, buying-group composition, or which channels have historically converted. This phase replaces the static channel list with an AI recommender that uses richer per-account signal, while keeping the template as a guaranteed-safe fallback.

- **New nodes (Brief Approval Handler):** `Gather Channel Context` (Postgres — account score, last 8 prior actions, top-8 channel-performance rows) → `Build Channel Context` (Code — assembles a human-readable prompt payload: industry/theme/buying-group/urgency/stage plus the matched template's own steps as JSON) → `Channel Recommendation AI` (langchain agent on `OpenAI Chat Model — Channel Reco`, gpt-5-mini) → `Parse AI Channel Plan` (Code — validates and sanitizes the AI's output). `Recommend Action` now builds its plan from `Parse AI Channel Plan`'s output instead of reading the template directly.
- **Model choice:** gpt-5-mini over the gpt-4o-mini used elsewhere in this project (e.g. the Brief Writer) — at the time this node was built, the n8n tooling's own model-selection guidance flagged gpt-4o/gpt-4/gpt-3.5 as superseded for *new* nodes. Existing nodes using gpt-4o-mini were left alone (not a regression to fix, out of scope here).
- **Guardrails, in order:** the prompt restricts the model to the existing fixed 10-action_type vocabulary and a 2–6 step plan; `Parse AI Channel Plan` then independently re-validates every `action_type` against that vocabulary and `delay_days` against a 0–60 range, caps at 8 steps, and — if the AI call errored, or the output is missing/unparseable/empty after validation — falls back wholesale to the matched `campaign_templates` row's original steps, unmodified. The cadence is structurally guaranteed to be valid either way.
- **Schema (migration `add_channel_recommendation_ai_columns`):** `actions` and `cadence_steps` both gained `channel_source text default 'template' ('ai'|'template')` and `channel_reco_reason text`, so every step's provenance and the AI's stated reasoning (when used) are queryable after the fact. `dashboard_cadence_steps` view updated to surface both; `pipeline-control-tower.html/-shared.html`'s Cadence steps table gained "Channel"/"Why" columns.
- **Caught during end-to-end testing, not during build:** the first real test execution (disposable account `ZZZ-QA-CHANNEL-AI-TEST`) hit two issues:
 1. The model's first response came back as syntactically invalid JSON (a missing closing brace on one step object) because the chat model node was left in default text mode. `Parse AI Channel Plan`'s try/catch correctly caught the parse failure and fell back to the template's steps — the run completed successfully and the safety net was proven to work for real, not just in theory. Fixed properly by switching `OpenAI Chat Model — Channel Reco`'s `options.textFormat.textOptions.type` to `json_object`, which constrains the model's output grammar so it can't produce malformed JSON. Re-ran: clean 5-step AI plan, `used_ai: true`, real DB rows with `channel_source='ai'` and per-step `channel_reco_reason`, and the parallel Salesforce Campaign sync still completed (`synced`) unaffected.
 2. **Found, not fixed — pre-existing, unrelated to this change:** the *existing* `Select Campaign Template` node (built 2026-06-28, untouched here) throws "there is no parameter \$4" whenever any one of `when_urgency/coverage_label/when_stage/trigger_type` is null on

the approved brief — its `queryReplacement` expression's `|| ''` fallback produces an empty segment that appears to get dropped when n8n splits the comma-joined string into positional parameters, shrinking the array below what the SQL expects. Confirmed present identically in both the draft and already-published active version of Brief Approval Handler — a live production risk for any brief with a genuinely null value in one of those four fields, not something introduced by this phase. Worked around for testing by giving the disposable test row a non-null `trigger_type`. **Needs a real fix** (e.g. `coalesce($1::text, '')` inside the SQL itself rather than `|| ''` in the expression that gets string-joined) — left for a follow-up, flagged here so it isn't lost.

New subsystem: Adaptive Decision Engine (Phase 4, 2026-06-29)

Every other phase so far automates *getting* a cadence built and executed; nothing yet reacted to how a step actually performed once it was measured — a campaign that already converted kept running its remaining steps, dead campaigns kept retrying the same channel forever, and warm-but-unconverted leads sat on whatever channel the template/AI happened to pick first. This phase closes that loop with a small, explicit rule engine over `outcomes`.

- **New, standalone workflow** rather than a modification of Action Outcome Measurement's existing branches — chosen to avoid regression risk on a live production workflow and the cross-branch item-reference ambiguity that would come from reading another branch's lineage mid-loop. **Adaptive Decision Engine** (`tGYW1gMBXQrC3bQ6`, daily 06:15, 15 minutes after Action Outcome Measurement runs): `Get Recently Measured Steps` (Postgres — outcomes measured in the last day, joined back to their `cadence_steps/campaigns`, filtered to `cs.adaptive_action is null` so nothing is ever re-adjudicated) → `Loop Measured Steps` (`splitInBatches`, size 1) → per step: `Get Campaign Signal` (Postgres — remaining pending steps, the next pending step's id/channel, whether a high-touch channel has been used yet, total engagement-event count for the campaign) → `Decide Adaptive Action` (Code — pure rule evaluation, builds a raw multi-statement SQL string) → `Apply Adaptive Decision` (Postgres — executes that SQL) → next batch.
- **Four rules, in priority order:** (1) **stop** — if the step's outcome was `met`, cancel every remaining `pending/recommended` step on the campaign and mark the campaign `completed/completion_reason='goal_met'`; goal achieved elsewhere makes further cadence steps wasted outreach. (2) **pause** — if the campaign has zero `engagement_events` and there's a next pending step, push that step's `scheduled_for` out 7 days; no signal at all yet means hitting the same dead lead on schedule is unlikely to help. (3) **escalate** — if there's been engagement but no conversion and the next pending step is on a low-touch channel (email/content_download/landing_page/ads/webinar/linkedin_engage), bump that step to a higher-touch channel — `linkedin_connect` if no high-touch channel has been used yet on this campaign, else `phone`; warm-but-stalled leads get a more personal channel instead of repeating what already isn't converting. (4) **reviewed** — none of the above; no-op, just stamped so it isn't re-evaluated. Every branch stamps the *originating* step's new `cadence_steps.adaptive_action/adaptive_reason` columns, which is what makes rule (1)'s filter idempotent without a separate tracking table.
- **Deliberately additive, not invasive:** pause/escalate work by mutating the *next* step's `scheduled_for/action_type` — Cadence Scheduler's existing `status='pending'` and `scheduled_for <= now()` query picks up both changes automatically, so zero changes were needed to Cadence Scheduler itself.

- **Schema (migration `phase4_adaptive_decision_columns`):** `cadence_steps` gained `adaptive_action text/adaptive_reason text`; `campaigns` gained `completion_reason text`. Note: the column comment on `adaptive_action` only lists `escalated/paused/stopped` — the fourth real value, `reviewed`, was added to the code after the comment was written and the comment hasn't been corrected yet (cosmetic, not a behavior bug).
- **Verification:** built 4 disposable test campaigns (`TEST_PHASE4_A-D`, one per rule branch, via direct SQL inserts — campaign → action → cadence_steps → outcome → engagement_events as needed), ran the published workflow once via `execute_workflow` (execution succeeded), then independently queried Postgres post-run state for all 4 campaigns and confirmed exact expected mutations: A's remaining steps cancelled + campaign completed/goal_met, B's next step pushed 7 days, C's next step's `action_type` swapped to the expected escalation target, D stamped `reviewed` with no other change. Re-ran the source query afterward and got 0 rows back, confirming the `adaptive_action is null` filter makes re-runs a no-op. All 4 test campaigns and their dependent rows deleted afterward; verified zero residual under the `TEST_PHASE4_%` tag.
- **Dashboard:** `dashboard_cadence_steps` view and both `pipeline-control-tower.html/-shared.html` Cadence steps tables gained "Adaptive" (action) / "Adaptive reason" columns, same pattern as Phase 5's Channel/Why columns.
- **Verification:** two full `execute_workflow` (manual mode — real credentials, real OpenAI call, real DB writes, no publish required) runs against the disposable test account, the second confirming the JSON-mode fix; DB rows for both runs inspected directly via `execute_sql` and the test campaign/action/cadence_steps/pipeline_runs rows deleted afterward (verified zero residual). Published to make this live in production after the second run came back clean.
- **Follow-up, same day:** `dashboard_channel_source_performance view`. Added to answer "did the AI's channel picks actually outperform the template's?" — same shape as `dashboard_hypothesis_winrate` but grouped by `channel_source` instead of `action_type` (executed/measured/met/win_rate_pct). Added now rather than later specifically *because* the system is still pre-traffic (`outcomes`: 0 rows, only 3 of 50 actions ever `executed`) — confirmed empty/near-empty on creation as expected, but it's wired into both dashboards and ready the moment real executed/measured actions accumulate, rather than a forgotten TODO. Surfaced as a new chart ("AI vs. template channel picks — win rate by source") on both `pipeline-control-tower.html` and `-shared.html`, right after the existing channel-performance chart.

Lead / Campaign Member sync — hardened, re-enabled, and published (2026-06-29)

User reframed priorities directly: the 10-phase roadmap is secondary to the actual goal — automate Salesforce lead creation/maintenance and get feedback. Picked back up the 5 Salesforce-writing nodes in Action Approval Handler that were disabled 2026-06-28 pending review (see above) and re-tested for real this time, rather than re-enabling on the strength of the original (simulated-pin-data) test alone. That decision paid off — live execution against real Salesforce surfaced two structural bugs the original build/test pass had no way to catch:

- **Found: Find Existing Lead's SOQL search/query operation does not loop per input item.** Unlike most n8n nodes, this Salesforce operation runs once per node-execution regardless of how many items are fed in — with 2 confirmed buying-group contacts in one item batch, it ran once, collapsed to a single empty placeholder output item (`alwaysOutputData:true`), and `pairedItem` referenced both original items at once. Anything downstream assuming per-contact search results silently got garbage.

- **Found: Create Lead read company/lastname/firstname/title/focus_area from the wrong node.** It used bare `$json.*`, which after `Find Existing Lead` only carries SOQL result columns (`Id`) — never the original contact fields. Confirmed via a real Salesforce API error: `"Required fields are missing: [LastName, Company]"`. This means the "create new Lead" path had likely never actually worked, even when the 5 nodes were "tested" before being disabled — pinned/simulated test data can't catch a wrong-node field reference like this.
- **Fix: wrapped the branch in a `splitInBatches` loop** (`batchSize:1`, new node "Loop Contacts" between `Extract Confirmed Contacts` and `Find Existing Lead`; loop body's two terminal nodes — `Add Existing Lead To Campaign` and `Add New Lead To Campaign` — wired back into it to pull the next contact). Copied this pattern directly from the Adaptive Decision Engine's proven "Loop Measured Steps" node rather than inventing a new shape. Also fixed `Create Lead`'s 5 fields to read `$( 'Extract Confirmed Contacts' ).item.json.*` instead of `$json.*` — the same explicit-node-reference pattern `Add Existing/New Lead To Campaign`'s `campaignId` already used correctly.
- **Found and closed a second idempotency gap.** The existing guard (`Mark Executed`'s `WHERE ... AND status <> 'executed'`, added when this branch was first built) correctly stopped the Lead-sync side on a replayed approve-link hit, but `Create a task` and `Send Action to Rep` sat on a separate, unguarded connection straight off `Mark Executed` — a replay still created a real (garbage-content) Salesforce Task and sent a duplicate rep email, because n8n's Postgres node emits a placeholder `{success:true}` item on a 0-row UPDATE rather than 0 items. Added a new `Action Updated? IF` node (checks `$json.id` notEmpty) gating both `Create a task` and `Extract Confirmed Contacts`, with the false branch routed to a new `Respond Already Processed` node instead of silently falling through.
- **Hardened the dedup match.** `Find Existing Lead`'s SOQL match grew from `LastName+Company` to `FirstName+LastName+Company`, closing a real collision risk (two different buying-group contacts sharing a surname at the same account would otherwise have collapsed onto one Salesforce Lead).
- **Verified end-to-end against disposable data** (`ZZZ-QA-LEADSYNC-TEST` — `pipeline_runs/campaigns/actions` rows, deleted after): one execution created two real, distinct Salesforce Leads (Jordan Vega, Taylor Vega) with correct field values; a second execution against a fresh action referencing the same two contacts correctly found and reused both existing Leads via the hardened dedup match instead of duplicating them; a replay of an already-executed action's approve link correctly short-circuited at `Action Updated?` with zero side effects (no duplicate Lead/Task/email). `onError: continueRegularOutput` on every Salesforce node in the branch was independently confirmed working — a deliberately-malformed test Salesforce Campaign ID caused expected, isolated `MALFORMED_ID` failures on `Create a task/Add Existing|New Lead To Campaign` without blocking `Send Action to Rep/Respond Approved`. Test Salesforce records (2 Leads, 1 Task) deleted afterward via a disposable probe workflow using the native `lead.delete/task.delete` operations, then archived.
- **Published.** Draft/active divergence checked and confirmed (active still had all 5 nodes disabled, no loop, no idempotency gate) before `publish_workflow` — consistent with every prior workflow edit in this project; see the repeated "Takeaway" note above.

Current state: Lead/Campaign Member sync is live in production, not just re-enabled but structurally fixed in ways the original build missed. Still open, unchanged from before: no real contact emails anywhere in `raw_payload.buying_group` (Lead `upsert` with a custom External ID field remains blocked on that), and `campaigns/actions` in production still have zero rows that have reached `executed` through this branch with a *real* (non-test) Salesforce Campaign — worth a spot-check the first time that happens organically.

New feature: Account 360 view + AI Decisions feed (2026-06-30)

Two views existed in the DB with no UI surfacing them at all: `channel_reco_reason` (Phase 5 channel-pick rationale, on `cadence_steps`) and `adaptive_reason` (Phase 4 stop/pause/escalate rationale). User asked for two things on top of existing data, with an explicit constraint to keep the existing dashboards exactly as informative as they already were — additive only, nothing removed or restructured:

- **Account 360** — new tab, account picker (datalist autocomplete off `dashboard_accounts_list`) or click any account name anywhere in the app (Briefs, Actions, Campaigns tables all got their Account cells turned into clickable links via a new `acctLink()` helper + Grid.js `formatter` columns). Shows score/engagement summary, campaigns with Salesforce sync status (new `sfPill()` helper for synced/error/skipped/pending), full AI-reasoning history for that account (`dashboard_ai_decisions` filtered), action/outcome history, rejection history, and engagement timeline.
- **AI Decisions** — new tab, reverse-chronological feed across all accounts of every AI channel pick and adaptive action with its plain-English reason, sourced from a new `dashboard_ai_decisions` view (unions `channel_reco_reason` and `adaptive_reason` into one `decision_type/reason` shape).
- Both tabs handle the current pre-traffic state gracefully: `dashboard_ai_decisions` returns 0 rows by correct design right now (0 `channel_source='ai'` actions executed yet, 0 `cadence_steps`, only 2 null `completion_reasons`) — empty state reads "No AI channel picks or adaptive actions yet — these populate once campaigns and outcomes start flowing" rather than looking broken.
- New view followed the project's standard `REVOKE ALL ... GRANT SELECT` pass (see the recurring default-ACL leak noted in the priority list below).
- Mirrored into both `pipeline-control-tower.html` and `-shared.html` — verified the shared copy's `<script>` block is byte-for-byte identical to the main file's (same `node --check`-validated JS), so the only remaining difference between the two files is still just the 4-line Settings-tab cosmetic difference noted below. All pre-existing tabs, charts, and tables in both files are unchanged.

New subsystem: Salesforce Opportunity creation, human-gated (2026-06-30)

The Adaptive Decision Engine's "stop" rule (Phase 4, 2026-06-29) already detects the moment a campaign hits its goal (`status='completed' AND completion_reason='goal_met'`) but did nothing with that signal beyond halting the cadence. This phase closes the loop: a goal-met campaign now triggers a human-approved Salesforce Opportunity, rather than either auto-creating one (no human check before a real CRM write) or leaving the signal to die on the dashboard. Followed the same HMAC-signed email-link approval pattern used everywhere else in this project (Brief Approval Handler, Action Approval Handler, Cadence Scheduler) rather than adding a dashboard write button — the dashboard is 100% read-only Grid.js-over-Postgres-views today, and a write path there would have been a materially larger architectural change than reusing a proven pattern.

- **Schema:** `campaigns` gained `opportunity_status` (`not_offered/offered/created/declined/error`, default `not_offered`), `opportunity_offered_at`, `opportunity_created_at`, `opportunity_error`, `salesforce_opportunity_id`.
- **New workflow: Opportunity Offer** (`dnGpK2weiPIcikGa`, daily 06:45 — 30 min after the Adaptive Decision Engine's 06:15 run, so newly-stopped campaigns are picked up same-day). Finds `goal_met/not_offered` campaigns, marks them `offered`, and emails an HMAC-signed approve/decline link pair to the rep.
- **New workflow: Opportunity Approval Handler** (`er7QvTbfSkyc60wx`, webhook at `/webhook/opportunity-approval`). Verifies the HMAC signature, then on approve: idempotency-

guarded UPDATE (WHERE opportunity_status='offered') → Create Salesforce Opportunity (Salesforce node, onError: continueRegularOutput so a CRM failure doesn't strand the approval) → writes salesforce_opportunity_id back on success or opportunity_status='error'+opportunity_error on failure. On decline: same idempotency-guarded UPDATE to declined, no Salesforce write at all.

- **Dashboard:** additive-only oppPill() helper + a status segment appended to the Account 360 campaign row template on both pipeline-control-tower.html and -shared.html — surfaces opportunity_status and, on error, opportunity_error. No new write affordance; purely a read-only indicator.

Three bugs found and fixed during build/test, none of them in the core design:

1. **create_workflow_from_code** silently ignored an explicit newCredential('Salesforce account 2') and wired the dangerous real-org credential ("Salesforce account") instead — twice, once on Create Salesforce Opportunity and again on a disposable cleanup workflow built later to delete test data. Both caught by reading autoAssignedCredentials in the tool's own response and cross-checking against the known-safe sandbox credential ID (HPnx6HXN8COY4aOe), fixed via update_workflow's setNodeCredential operation before any execution. Documented as a standing risk in memory (feedback_n8n_credential_autoassign_risk) — this tool cannot be trusted to honor a named credential, full stop; always re-verify.
2. **update_workflow's setNodeParameter (JSON-Pointer path)** silently corrupted a node instead of erroring when the path targeted an array element nested under a sibling key sharing the node's own parameters name — it created a spurious nested parameters.parameters.conditions.conditions.{"0":...} object instead of editing the real conditions.conditions[0].rightValue, and reported success despite not fixing anything (the next test run reproduced the original error unchanged). Root cause: array-index path segments under setNodeParameter are unreliable for this node shape. Fixed by switching to updateNodeParameters with replace: true and the full corrected parameters object — a whole-object replace sidesteps path-segment ambiguity entirely. Worth defaulting to updateNodeParameters(replace) over setNodeParameter for any nested array edit going forward.
3. **Real Salesforce sandbox-org schema gap:** the Create Salesforce Opportunity node's additionalFields.campaignId failed with INVALID_FIELD — No such column 'CampaignId' on subject of type Opportunity — this sandbox org doesn't have Campaign Influence enabled, so the standard CampaignId field isn't API-accessible on Opportunity at all, independent of the value sent. Dropped the field; the originating Salesforce Campaign ID is still recorded in the Opportunity's description text instead.

Verified end-to-end against disposable data (ZZZ-QA-OPP-APPROVE/ZZZ-QA-OPP-DECLINE, campaign ids 12/13, deleted after): Opportunity Offer's discovery query, status update, HMAC-signed email build, and real Gmail send all worked correctly on the first run, no bugs found in that workflow. Opportunity Approval Handler: approve created a real sandbox-org Opportunity (006d500000AUCq5AAD) with the correct salesforce_opportunity_id written back to Postgres; decline recorded declined with no Salesforce write; replaying both the approve and decline links after they'd already been processed correctly short-circuited to "already processed" via the idempotency guard; an invalid signature was correctly rejected with a 403. Cleanup: the test Opportunity was deleted from the sandbox org via a disposable one-off workflow (immediately archived after), and both test campaigns rows were deleted from Postgres — verified zero residual. Both workflows published and live.

New subsystem: Salesforce Activities read-back (2026-06-30)

Closes a gap on the user's own self-authored roadmap: Salesforce was a one-way sync (Supabase → Salesforce, Leads/Campaigns/Tasks created but never read back) and there was no path for what reps actually do on a Lead in Salesforce — logging a call, a note, a follow-up Task — to flow back into this system's measurement layer. Reusing the same "store the Salesforce ID, then read against it" pattern already proven for Campaign sync (the synced/error/skipped/pending pill on Account 360): a new `salesforce_leads` mapping table records every Lead this pipeline creates or matches, and a new daily job reads Salesforce Tasks/Events logged against those Leads and inserts them into `engagement_events` as a new `crm_activity` source type. This also folds in idea #8 (SMS/WhatsApp capture) for free — reps already log activity in Salesforce when they work a Task, so no new channel/API integration was needed to start capturing it.

- **Schema:** new table `salesforce_leads` (`account`, `contact_name`, `salesforce_lead_id` unique, `campaign_id` FK, `created_at`) — the mapping table this whole subsystem reads against. `engagement_events` gained `salesforce_activity_id` `text` (dedup key for the read-back insert, mirrors the existing `WHERE NOT EXISTS` idempotency pattern used by the GA4 feeder and Engagement Ingest).
- **Action Approval Handler gained two new nodes** on its existing Lead-sync branch (the one hardened 2026-06-29, see above): **Record Lead Mapping (Existing)** (Postgres insert, sits between `Lead Found?`'s true branch and `Add Existing Lead To Campaign`, reads the matched Lead's `Id` from `Find Existing Lead`'s SOQL output) and **Record Lead Mapping (New)** (same insert shape, sits between `Create Lead` and `Add New Lead To Campaign`, reads the newly-created Lead's lowercase `id`) — both `WHERE NOT EXISTS`-guarded on `salesforce_lead_id`. This is the only change to that workflow; everything else in the Lead-sync branch is unchanged from the 2026-06-29 hardening pass.
- **New workflow: Salesforce Activities Read-back** (`mL0H9QHe73SouLsR`, daily schedule). `Get Active Leads` (Postgres, all `salesforce_leads` rows) → `Build SOQL Queries` (Code, builds a `WhoId IN (...)` SOQL filter plus a `leadMap` keyed by Salesforce Lead ID for downstream lookup) → `Query Tasks` + `Query Events` (Salesforce SOQL search, parallel) → `Combine Activities` (Merge, append) → `Map To Engagement Event` (Code, joins each Task/Event back to its `salesforce_leads` row via `leadMap`, building `account/contact/occurred_at/metadata` — drops any activity whose `WhoId` isn't a tracked Lead) → `Insert Engagement Event` (Postgres, `source='salesforce'`, `event_type='crm_activity'`, `WHERE NOT EXISTS` on `salesforce_activity_id`).

Two Code-node bugs found and fixed during build/test, both the same root cause — an n8n Code node's `mode` parameter is not safely inferred and must always be set explicitly:

1. **Build SOQL Queries silently processed only the first input item.** Built with no explicit `mode`, it behaved like `executeOnce`/per-item mode rather than aggregating — with 2 Leads fed in, `leadMap` only ever contained the first one, so any activity belonging to the second Lead would have been silently dropped from the SOQL filter entirely (not even queried, let alone matched). Fixed by explicitly setting `mode: 'runOnceForAllItems'` and looping over `$input.all()` in the code.
2. **Map To Engagement Event had the identical bug, independently** — also built with no explicit `mode`, also silently dropped a valid second item (confirmed with a 3-item merged Task/Event test batch: only the first survived). Same fix: explicit `mode: 'runOnceForAllItems'` + manual loop. Caught both times via `test_workflow` + `get_execution` item-count inspection, not by the tool surfacing an error — an unset `mode` fails silently, it does not fail loudly. New standing rule going forward: every Code node in this project must set `mode` explicitly; never rely on whatever n8n's implicit default does.

Verified end-to-end against real sandbox data, not just test_workflow pin data (using `execute_workflow`'s `executionMode: 'manual'`, which runs the current draft with real credentials without requiring publication — discovered this run, since the workflow's only trigger is a Schedule node with no manual/webhook alternative, and `publish_workflow` separately confirmed a manual-trigger-only workflow can never be activated, "at least one trigger, webhook, or polling node is required"):

- A disposable QA workflow created one real sandbox Lead and one real sandbox Task (ZZTEST Readback Co); a `salesforce_leads` row was inserted pointing at the Lead. Running the Activities Read-back workflow live correctly found the Task via SOQL, mapped it through `leadMap`, and inserted the correct `engagement_events` row (`source='salesforce', event_type='crm_activity', full metadata`). Running it a second time confirmed the `WHERE NOT EXISTS` guard correctly deduped — no second row.
- Separately, exercised Action Approval Handler's two new `Record Lead Mapping` nodes live via `execute_workflow` with a simulated webhook GET (`type: 'webhook', webhookData.query` carrying a real, locally-HMAC-signed `id/action/sig`) against disposable `pipeline_runs/campaigns/actions` rows — one run matched the existing test Lead (`Lead Found? true` branch → `Record Lead Mapping (Existing)`), a second run with an unseen contact name correctly created a new Salesforce Lead (`Lead Found? false` branch → `Create Lead` → `Record Lead Mapping (New)`). Both branches inserted correct `salesforce_leads` rows with the right Lead ID. A deliberately-invalid `campaigns.salesforce_campaign_id` test value caused the expected, isolated `MALFORMED_ID` failures on `Add Existing/New Lead To Campaign/Create a task` without affecting the Lead-mapping inserts or the rep notification email — consistent with the existing `onError: continueRegularOutput` pattern on that branch.
- Cleanup: all disposable Postgres rows (`salesforce_leads, engagement_events, actions, campaigns, pipeline_runs`) deleted and individually re-verified at 0 each; the real sandbox Salesforce Task and both sandbox Leads (one pre-existing test Lead, one newly created during the second branch test) deleted via a disposable cleanup workflow; both disposable QA workflows archived after use. Zero residual test data left in Postgres or Salesforce.
- Sixth confirmed occurrence of the `create_workflow_from_code` credential auto-assign bug (on the disposable cleanup workflow's three Salesforce delete nodes) — caught via `autoAssignedCredentials` and fixed via `setNodeCredential` before execution, per the standing rule in `feedback_n8n_credential_autoassign_risk`.

Both the Activities Read-back workflow and the Action Approval Handler draft (carrying the two new Lead-mapping nodes) are now published and live.

Coworker pressure-test fixes: template audit trail, hard/soft scoring, SF Campaign write-back, diminishing returns (2026-06-30)

A coworker (Jocelyn) pressure-tested the pipeline end to end and surfaced a 6-item gap list. Items 5 (Lead/Campaign Member sync) and 6 (a non-technical item) were already covered or out of scope; this pass closed the remaining four:

- **Item 1 — template-selection audit trail.** `campaigns` gained `template_match_trace jsonb`. Brief Approval Handler's `Select Campaign Template` query now returns every `campaign_templates` row that matched the brief (not just the winner), ranked by `priority`; a new code node, `Build Template Match Trace`, reduces that ranked list to `{matched_count, winner_key, winner_name, winner_priority, runner_ups, reason}` and `Insert Campaign` writes it alongside the existing

`template_key`. `template_key` stays "what we picked"; `template_match_trace` is "why," including every runner-up that also matched and lost.

- **Item 2 — hard vs. soft signal separation.** `account_score` (view) now exposes `hard_points` and `soft_points` alongside the existing blended `score`: `hard_points` sums only events whose `engagement_events.source` is `ga4`, `gmail`, or `salesforce` (independently observed); `soft_points` sums only `source='manual'` (self-reported). Brief Approval Handler's `Gather Channel Context` query and `Build Channel Context` code node now pass both numbers to **Channel Recommendation AI**, whose system prompt was updated to explicitly weight hard signals more heavily than soft ones when picking a channel sequence.
- **Item 4 — diminishing returns on repeated soft signals.** `account_score`'s event-scoring CTE gained `occurrence_rank` (`row_number()` OVER (PARTITION BY `account`, `event_type` ORDER BY `occurred_at`)): the 1st occurrence of a given event type counts at full weight, the 2nd at 0.4x, the 3rd and beyond at 0x. Without this, a high volume of low-effort events (e.g. repeated `page_visit/email_open`) on one account could inflate its score without bound; now it plateaus.
- **Item 3 — Salesforce Campaign status write-back.** The Salesforce Campaign created at brief-approval time (`salesforce_campaign_id`) was previously written once (`Status='In Progress'`) and never updated again, even after the campaign completed or was aborted. Adaptive Decision Engine's `Decide Adaptive Action` code node now also computes `salesforce_status` (`completed+goal_met` → `Completed`; `completed+any other reason` → `Aborted`; otherwise unchanged/null) alongside its existing stop/pause/escalate logic; a new `Has Salesforce Campaign?` IF node gates a new `Update Salesforce Campaign Status` Salesforce node on both `salesforce_status` and `salesforce_campaign_id` being present.
- **Dashboard:** additive only, both `pipeline-control-tower.html` and `-shared.html` (verified byte-identical for these additions). Account 360's score KPI card now shows `Hard: N · Soft: N` alongside the blended score; the campaign row gained `sfStatusPill()` (client-side mirror of the same `completed/goal_met` → `Completed/Aborted/In Progress` mapping, so the dashboard reads correctly even before the next Adaptive Decision Engine run actually pushes the Salesforce update) and `templateTraceNote()` (renders the winning reason plus any runner-up template names/priorities inline).

One bug found and fixed, affecting both items 1 and 2: an editing pass on `Select Campaign Template` and `Gather Channel Context` (both Postgres nodes) used a path-based parameter edit that created a stray **nested duplicate key** — `parameters.parameters.query` — sitting alongside the real top-level `parameters.query` n8n actually executes. `get_workflow_details` still *looked* right at a glance (the new SQL was present in the JSON somewhere), but the live node kept running the old, pre-fix query underneath — silently making the item 1 and item 2 schema/SQL changes inert. This is a new sibling of the previously-documented `setNodeParameter`-on-array bug (`feedback_n8n_setnodeparameter_array_bug`): same failure shape (the tool reports success, the edit doesn't error, but it also doesn't take effect), different node/field type. Caught by re-diffing the full node JSON rather than trusting a partial view, and fixed by switching to `updateNodeParameters` with `replace: true` (whole-object overwrite) on both nodes — the same remedy already adopted for the analogous IF-node bug found during the Opportunity-creation build. New standing rule, now confirmed twice: prefer `updateNodeParameters`(`replace`) over any path-based parameter edit on an existing node, and always re-diff the *complete* node JSON after an edit instead of spot-checking the field you changed. Documented in `feedback_n8n_nested_duplicate_parameters_bug`.

Verified, not via a live webhook test. `test_workflow` pins every credentialed node (Postgres, Salesforce, the AI agent) to simulated data even during a "test" run, so it can't actually exercise a live SQL fix — and an

earlier segment separately found that `test_workflow`'s pinning can corrupt `pairedItem`-based cross-node references through a pinned multi-run node, producing wrong branch results that look like a real bug but are actually a test-tooling artifact (`feedback_n8n_pairedItem_pinned_node_fragility`). Given that, items 1/2/4 were verified by running the corrected SQL directly against live (read-only) Supabase data — confirmed `Select Campaign Template`'s new query returns all matching templates ranked by priority (4 rows for a sample brief, not just the winner), confirmed `Gather Channel Context`'s new query returns correct `hard_points/soft_points` for a real account, and confirmed the live `account_score` view definition has the intended `occurrence_rank` decay and hard/soft filter logic — plus manual tracing of `Build Template Match Trace/Build Channel Context`'s code against that real data. Item 3's logic was verified by code review of `Decide Adaptive Action`'s new branch against the same rule table used for the original stop/pause/escalate logic (already live-tested when Phase 4 shipped). Both workflows' drafts were then published — confirmed via `get_workflow_details` that `versionId === activeVersionId` on both.

4. Dashboards

Both `pipeline-control-tower.html` and the `-shared.html` copy are otherwise identical — same Chart.js/Grid.js setup, same Supabase views queried, same tabs (Dashboard, Briefs, Actions, Campaigns, Account 360, AI Decisions, Settings). The only difference is the 4 lines covering the Settings tab (editable vs. disabled inputs + missing Save button), which, as covered above, is a cosmetic difference only — not a real security boundary.

Priority list

1. **Security — Done 2026-06-28** (see remediation note above): RLS + grants locked down on the 7 exposed tables, `app_config` anon UPDATE scoped to the 2 Settings-tab keys, approval webhooks now HMAC-signed. Kept the "anon key in static HTML" + no-login model rather than adding an auth layer — that remains the accepted trust model for this internal tool, just no longer paired with wide-open tables and unauthenticated approval links.
2. **~~Brief-approval emails are currently broken~~ — Fixed 2026-06-30**. The live/active "Intelligent Automation — Delivery Layer" (`HQdvWtRfLdzDTN3X`) was building approve/reject links via `Edit Fields5` (plain `?id=...&action=...`, no signature) before `Approval Email`, while "Brief Approval Handler" (`AoVkl0ncTxZqQlwz`) requires a valid HMAC on every `/webhook/brief-approval` call and 403s otherwise. A draft already had the fix built (`Sign Brief Links` node, same HMAC scheme, correctly wired `Insert rows in a table` → `Sign Brief Links` → `Approval Email`, old `Edit Fields5` already removed) — the only blocker was `Approval Email` being disabled in that draft. Fix: re-enabled `Approval Email` (`update_workflow/setNodeDisabled`), then published. Verified end-to-end with disposable data ("ZZZ QA Approval-Fix Test Co", `pipeline_runs.id=394`): a real `execute_workflow` run on the published Delivery Layer produced a signed reject link (`...&sig=9895...`), and feeding that exact link into Brief Approval Handler returned `valid:true` from `Verify Signature` and routed through `Signature Valid?` to `Respond Rejected` — no 403, signature accepted. Test row deleted afterward.
3. **`app_config.tracking_base_url` and `app_config.when_engine_url` are empty — blocks two subsystems, found 2026-06-30**. Brief Approval Handler's `Recommend Action` falls back to the literal string `https://example.com` for every tracked URL it builds when `tracking_base_url` is blank (confirmed in the node's code) — every UTM-tracked link this pipeline has ever generated is a placeholder, not a real destination, until this is set to the actual marketing site's base URL. Separately, `when_engine_url` blank means the `Engine URL set?` IF node always routes to "no" — rejected-brief

feedback meant for the WHEN engine is silently dropped rather than forwarded. Both are one-line `app_config` UPDATES once real values exist; until then, no tracked click data and no reject-feedback loop can ever reach their destinations, which is also why `engagement_events/outcomes` have stayed empty.

4. ~~Swap the GA4 workflows~~ — **Already done 2026-06-28** (re-confirmed live 2026-06-30): "GA4 Engagement Connector" is inactive, "GA4 → Engagement Feeder" is active — this item was actually closed the same day it was opened (see `n8n-workflows/README.md`'s 2026-06-28 note); this list just never got updated to reflect it.
5. **Refresh documentation** — `supabase/schema.sql` re-synced 2026-06-30 (`campaigns.template_match_trace`, `account_score`'s full hard/soft-split + diminishing-returns rewrite, `dashboard_top_accounts/dashboard_campaigns` picking up the new columns, plus the three Account 360 views that existed live with no local definition at all: `dashboard_accounts_list`, `dashboard_account_actions`, `dashboard_ai_decisions`). `n8n-workflows/README.md` now also documents Cadence Scheduler, Adaptive Decision Engine, Opportunity Offer/Approval Handler, and Salesforce Activities Read-back, and its "Intelligent Automation (monolith)" row no longer claims that workflow is active (corrected 2026-06-30 — see item 6). Still open: a decision on the undocumented vector/RAG tables (`documents`, `agent_inputs`, `agent_inputs_bak`) — document or drop. Given how often `schema.sql` has drifted from live (this is the third time it's needed a sync pass), worth treating "did this change touch `schema.sql`?" as a standing checklist item on every future Supabase change, not a periodic catch-up.
6. **Housekeeping — updated 2026-06-30**. The 3 duplicate "Gate 1" workflows are confirmed gone from n8n entirely (0 results on `search_workflows`, not just inactive) — that part of this item is done, no action needed. Newly found: "Intelligent Automation" (`rNFL6JkvW5TxnaQv`, the monolith README listed as "still active") is actually **inactive with 0 executions ever** — its node names (`Signal Collector`, `Disqualifier`, `Criteria Setter`, `Judge`, `Mock Gate 2 Set Node`) match the original Gate1/Gate2 prototype era, fully superseded by Delivery Layer + Brief/Action Approval Handler; safe to archive, README corrected in the meantime. Also newly found: `campaigns` has 2 leftover disposable rows ("ZZZ QA Recheck Co", `salesforce_campaign_id='ZZZQARECHECK_FAKE_CAMPAIGN_ID'`, created 2026-06-29) that were never deleted after a QA pass — meaning there are currently **zero real production campaigns** in the table; delete both rows. Still open from before: "My workflow 2"; resolve or explain the 13 stale `pipeline_errors` (sampled rows are a mix of literal seed placeholders — "Example Workflow"/"Example Error Message" — and at least one real-looking unresolved error from a Disqualifier-node failure on 2026-06-11); fix the `rep@yourcompany.com` placeholder; clean up `*_bak` tables (`pipeline_runs_bak` 152 rows, `pipeline_errors_bak` 12, `agent_inputs_bak` 0, none with a PK) once confirmed unneeded. Also newly found via Supabase's performance advisor: 4 foreign keys have no covering index (`actions.campaign_id`, `campaigns.run_id`, `engagement_events.action_id`, `salesforce_leads.campaign_id`) — cheap to add, worth doing before real traffic arrives.
7. **Remove the default-ACL rule on public** — `ALTER DEFAULT PRIVILEGES FOR ROLE postgres IN SCHEMA public GRANT ALL ON TABLES TO anon`, `authenticated` has now caused the same auto-updatable-view leak 3 times (see security note above). Per-view revoke-then-grant fixes the symptom but not the cause; the next new view will leak again until the rule itself is dropped. **Confirmed 2026-06-30 via direct `pg_default_acl` query that this is broader than previously documented**: the standing rule grants `anon/authenticated` full `arwdDxtm` on future tables and `rwU` on future sequences — only functions are correctly locked to no-privileges by default. Any fix needs to cover both objtypes, not just tables.

8. **Fix ~~Select Campaign Template's "\$4 missing parameter"~~ bug — Fixed 2026-06-29.** Root cause confirmed empirically (not guessed): `queryReplacement`'s old form (`={{ expr1 }}, {{ expr2 }}, {{ expr3 }}, {{ expr4 }}`, each with an `|| ''` null fallback) is evaluated as one big string and then comma-split by `n8n` into the parameter array; when a field is null, the `|| ''` produces a zero-length segment that the splitter drops, shrinking the array below 4. Verified by reproducing the exact "**there is no parameter \$4**" error in a disposable isolated test workflow against the real Postgres credential (harmless `SELECT $1,$2,$3,$4` probe, no production tables touched), then confirming a single-expression array literal (`={{ [expr1, expr2, expr3, expr4] }}`) sidesteps the string-splitting entirely and returns all 4 params correctly, including empty ones. Applied that form to `Select Campaign Template` in Brief Approval Handler, diffed draft vs. active (only this one field changed, nothing else), and published. **Note (2026-06-30):** this same node's `query/queryReplacement` was edited again for the template-audit-trail work (see the coworker pressure-test section above) and hit the nested-duplicate-parameters bug during that pass; re-confirmed after the fix that the live query still uses the array-literal `queryReplacement` form (no `$4` error) alongside the new multi-row ranking logic.